

Text, colors, and backgrounds

In the last lesson, you learned how selectors choose which elements a CSS rule applies to. Now you can use those selectors to change the most visible parts of a page: text, colors, and backgrounds.

This lesson uses one small page and styles it in stages, so each rule makes something visible happen in the browser.

Text and color styles answer practical questions:

- Is the page using a readable font?
- Does the heading feel more important than the paragraph?
- Is the message easy to notice?
- Does the text have enough contrast against its background?

Start with the page before styling

To follow along, use two files: `index.html` for the page and `style.css` for the styles. Keep `style.css` empty at first.

Start with the unstyled HTML:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Text and color practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <h1>Text and color practice</h1>
      <p class="lead">CSS can make text easier to read before you build any complex layout</p>

      <article class="article-preview">
        <h2>Why readable text matters</h2>
        <p>Most pages ask users to read, scan, compare, or decide. Font size, line height, and color make it easier to read, scan, compare, or decide.
        <p class="price"><span class="old-price">$49</span> $29 today</p>
        <a class="read-more" href="/learn-more">Read more</a>
      </article>

      <p class="notice">Email is required before you can continue.</p>
    </main>
  </body>
</html>
```



Open the page in a browser before adding CSS. It still shows the content, but it uses browser defaults for the font, text size, link color, spacing, and page background. That plain version is useful because it gives you a before picture.

As you read the next sections, add the CSS blocks that match this page to `style.css` and reload. Try to notice what changed before reading the explanation.

Add a base style

A simple `body` rule can change the feel of the whole page:

CSS

```
body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}
```

- `font-family` changes the typeface.
- `color` sets the default text color.
- `background-color` sets the page background.

`system-ui` tells the browser to use the operating system's normal interface font. The `sans-serif` part is a fallback. If the browser cannot use the first font, it tries the next one.

You can give the browser a list of fonts:

CSS

```
body {
  font-family: Arial, Helvetica, sans-serif;
}
```

Font names with spaces go in quotes:

CSS

```
body {
  font-family: "Times New Roman", serif;
}
```

You do not need to obsess over fonts while learning CSS.

 **Start with readable defaults** - A small `body` rule is often the first useful CSS on a page. Set the font, default text color, and page

background before styling individual pieces.

In the practice page, we will also keep the content from stretching too wide:

CSS

```
.page {  
  max-width: 720px;  
  margin: 40px auto;  
}
```

This is exercise scaffolding, not what the lesson is about. Lessons 5 and 6 teach `max-width` and `margin` properly. For now, these declarations help the sample page look readable while you focus on text and color.

Make headings and paragraphs readable

The most common text properties are `font-size` , `font-weight` , and `line-height` .

CSS

```
h1 {  
  font-size: 40px;  
  font-weight: 700;  
  line-height: 1.1;  
}  
  
h2 {  
  font-size: 28px;  
  line-height: 1.2;  
}  
  
p {  
  font-size: 18px;  
  line-height: 1.6;  
}  
  
.lead {  
  font-size: 20px;  
  color: #4b5563;  
}
```

`font-size` controls how large the text is. Beginners often start with `px` because it is easy to see: `16px` , `18px` , `40px` . You will also see `rem` and `em` later. They scale with font size and user preferences, which matters for accessibility. `px` is fine while you are learning the basics.

`font-weight` controls how thick the text is. Normal text is usually `400` , and bold text is often `700` .

`line-height` controls the space between lines of text. Body text usually needs more line height than a heading, and a paragraph with `line-height: 1.6` , for example, is often easier to read than one where the lines are packed tightly together.

The `.lead` rule makes the intro paragraph slightly larger and softer than the main body text. That tells the reader, "this paragraph introduces the page."

 **Line height helps readability** - If a paragraph feels cramped, try increasing `line-height` before changing lots of other styles. Text can look cleaner just by giving each line more room to breathe.

Set text and background colors

The `color` property sets the foreground text color:

CSS

```
p {
  color: #374151;
}
```

The `background-color` property sets the background behind an element:

CSS

```
body {
  background-color: #f9fafb;
}
```

The names are easy to mix up at first:

- `color` changes the text color.
- `background-color` changes the color behind the element.

You can use both on the same element:

CSS

```
.notice {
  color: #92400e;
  background-color: #fffbeb;
}
```

That kind of pair is useful for messages, notes, warnings, and status labels. The text color and background color should work together so the message is readable.

You can also give a section its own background:

CSS

```
.article-preview {
  background-color: white;
}
```

The white background separates the article from the softer page background. It may still feel unfinished because there is no padding around the content yet, and that is fine for now. Padding belongs to the box model, which is the next lesson.

You can also use image backgrounds:

CSS

```
.hero {  
  background-image: url("hero.jpg");  
}
```

Background images have more details: sizing, repeating, positioning, and contrast with text on top. You will learn those later, so for now focus on solid background colors. They are easier to control, easier to read, and easier to debug.

Align and decorate text when it helps

text-align controls horizontal text alignment inside a block. The common values are **left**, **right**, **center**, and **justify**.

CSS

```
h1 {  
  text-align: center;  
}
```

Try this on the practice page for a moment, then remove it if the page feels better left aligned. Centered text can work for short headings or intro sections, but long centered paragraphs are harder to read.

text-decoration can add or remove lines around text:

CSS

```
.read-more {  
  color: #2563eb;  
  text-decoration: none;  
}  
  
.old-price {  
  color: #6b7280;  
  text-decoration: line-through;  
}
```

Links are underlined by default in browsers. Removing the underline can be fine in navigation or repeated interface links, but make sure the link still clearly looks clickable through color, placement, or another visual cue. The **line-through** value is useful elsewhere, like striking out an old price.

⚠ **Do not hide links completely** - If you remove a link underline and the color looks like normal text, users may not realize it is clickable. Styling should make the page clearer, not more mysterious.

Recognize common color formats

CSS understands several color formats. You do not need to memorize all of them, but you should recognize the common ones.

Named colors are readable in tiny examples:

CSS

```
.error-message {  
  color: red;  
}
```

They are fine for tiny examples, but real projects usually use more precise color values.

Hex colors are more common in real projects:

CSS

```
.error-message {  
  color: #b91c1c;  
}
```

A hex color starts with `#`, followed by numbers and letters. You will see hex values in design tools, codebases, and documentation.

`rgb()` describes a color with red, green, and blue values:

CSS

```
.button {  
  background-color: rgb(37, 99, 235);  
}
```

`hsl()` describes a color with hue, saturation, and lightness:

CSS

```
.button {  
  background-color: hsl(221, 83%, 53%);  
}
```

You will probably use hex most often at first. The useful skill is not memorizing color math. The useful skill is recognizing that all of these are just different ways to give CSS a color.

Some color formats can include transparency too:

CSS

```
.highlight {  
  background-color: rgba(37, 99, 235, 0.12);  
}
```

The **a** in **rgba()** stands for alpha, which is the transparency value. The last value, **0.12**, means the color is mostly transparent: **1** is fully visible, and **0** is fully transparent.

Check contrast before calling it done

Color is not only decoration; it affects whether people can read and use the page.

Imagine a muted style pushed too far:

CSS

```
.too-faint {  
  color: #d1d5db;  
  background-color: #f3f4f6;  
}
```

The text and background are too close in brightness.

This version is easier to read:

CSS

```
.muted {  
  color: #4b5563;  
  background-color: #f3f4f6;  
}
```

The text is darker, so it stands out from the background.

You do not need to become a color expert right away. Start with a simple habit: if you set a text color, check it against the background it sits on.

 **Color should not be the only signal** - Do not rely on color alone to communicate meaning. If an error message is only red, some users may miss the signal. Use clear text too, such as "Email is required."

Try it

You already styled the page in stages. Before moving on, change a few visible details on purpose:

- Change **body** to use **font-family: Georgia, serif;**, then change it back to **system-ui, sans-serif**.

- Add `text-align: center;` to `h1` , reload, then remove it.
- Change the `h1 font-size` from `40px` to `32px` .
- Change paragraph `line-height` from `1.6` to `1.1` , then back to `1.6` . Notice how readability changes.
- Change the `.lead` color to `#9ca3af` . Decide whether it still has enough contrast.
- Change the `.notice` background color to `#fee2e2` and the text color to `#991b1b` .
- Remove the `background-color` from `.article-preview` and reload.

Then add a second message under the notice:

html

<p class="success-message">Profile saved successfully.</p>

Write a new CSS rule for `.success-message` . Give it a text color and a background color that feel different from the warning notice, while still being readable.

The article and notice may still look cramped because their backgrounds have no inner spacing yet. That is not a mistake; the next lesson explains how padding gives a background room around its text.

The goal is to connect CSS properties to visible changes. When you write a rule, reload the page and ask: what changed, and did it make the page easier to read?

< 0 / 10 >

Knew 0 Items

Learnt 0 Items

Skipped 0 Items

ResetProgress

Test Your Understanding

Why open the page before adding CSS?

[Click to Reveal the Answer](#)

✔ Already Know that

⚡ Didn't Know that

⏮ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →